LIBRARY MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

STUDENT NAME	STUDENT CODE	REGISTRATION NO.	ROLL NO.
RAHUL DAS	BWU/MCA/26/021	26102000139	26102410017

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



MAY 2026

LIBRARY MANAGEMENT SYSTEM

A Terminal-Based Object-Oriented Library Automation, Circulation & Financial Management System

Submitted by

STUDENT NAME	STUDENT CODE	REGISTRATION NO.	ROLL NO.
RAHUL DAS	BWU/MCA/26/021	26102000139	26102410017

Under the Guidance of

Mr. Rahul Kumar Ghosh

Assistant Professor, Department of Computational Sciences



DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY, KOLKATA

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

DEPARTMENT OF COMPUTATIONAL SCIENCES

BONAFIDE CERTIFICATE

Certified that this project report titled **"LIBRARY MANAGEMENT SYSTEM"** is the bonafide work of **RAHUL DAS (Student Code: BWU/MCA/26/021)** who carried out the project work under my supervision in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications (MCA)** at Brainware University.

SIGNATURE

Dr. Jayanta Aich
HEAD OF THE DEPARTMENT
Computational Sciences
Brainware University, Kolkata

SIGNATURE

Mr. Rahul Kumar Ghosh
PROJECT SUPERVISOR
Assistant Professor, Computational Sciences
Brainware University, Kolkata

EXTERNAL EXAMINER SIGNATURE

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to **Brainware University** for providing me with the opportunity and computing facilities to undertake this academic project titled **"LIBRARY MANAGEMENT SYSTEM."** This endeavor has offered me profound exposure to Object-Oriented Programming (OOP) paradigms, relational database persistence, clean architecture, and terminal interface engineering.

I extend my profound thanks to my respected Project Supervisor, **Mr. Rahul Kumar Ghosh**, Assistant Professor, Department of Computational Sciences, for his exceptional guidance, technical evaluation, and continuous encouragement throughout each development phase. His architectural recommendations in class modularization, custom exception hierarchies, and financial fine status modeling were indispensable.

I also convey my sincere appreciation to **Dr. Jayanta Aich**, Head of the Department of Computational Sciences, for fostering an academic environment that stimulates technological rigor, practical implementation, and student excellence.

Special thanks are due to all respected faculty members, laboratory instructors, and administrative staff of the department for their continuous cooperation and support.

Lastly, I express my gratitude to my family and fellow classmates for their constant moral support, which enabled the successful design, implementation, and rigorous verification of this project.

RAHUL DAS

Student Code: BWU/MCA/26/021

Department of Computational Sciences, Brainware University

ABSTRACT

Conventional library operations in educational institutions often rely on manual ledger books or rigid, fragmented desktop applications prone to data inconsistency, cumbersome manual calculation of overdue penalties, and memory-intensive identifier lookups. In standard command-line software, users are typically burdened with memorizing arbitrary numerical IDs prior to issuing or returning items, leading to severe operational friction and high error rates.

This project presents the **"LIBRARY MANAGEMENT SYSTEM,"** an enterprise-grade terminal-based circulation and inventory management application built strictly with **Python Object-Oriented Programming (OOP)** and backed by an embedded, transactional **SQLite3 relational database**. The system models core academic library entities through encapsulated classes (`Book`, `Student`, and `Library`), orchestrated by an autonomous `DatabaseManager` that features automated DDL schema migrations and foreign key constraints.

Key architectural innovations include: **(1) Contextual Entity Previews** that dynamically render filtered, aligned ASCII tables of available books, registered students, or pending dues directly above every input prompt, completely eliminating user ID memorization; **(2) A Three-State Financial Fine Engine** (`NONE`, `DUE`, and `PAID`) paired with an automated overdue penalty algorithm (₹5.00/day after a configurable 7-day lending window); **(3) Dual Issue History Streams** separating currently checked-out books with live overdue flags from comprehensive historical transaction logs; and **(4) Granular Financial Auditing** allowing administrators to track global dues, per-student outstanding balances, total revenue collected, and payments by individual members.

The software was validated through an automated test suite comprising **54 unit tests** across 7 test suites with 100% pass rates, confirming sub-millisecond query execution, strict relational integrity, and robust exception handling for real-world institutional deployment.

Keywords: Library Management System, Object-Oriented Programming (OOP), SQLite3 Database, Circulation Engine, Overdue Fee State Machine, Contextual Terminal UI, Automated Unit Testing.

TABLE OF CONTENTS

Chapter	Title	Page No.
	ABSTRACT	iii
	LIST OF TABLES	v
	LIST OF FIGURES	v
	LIST OF SYMBOLS & ABBREVIATIONS	vi
1.	**INTRODUCTION**	1
2.	**OBJECTIVES**	1
3.	**PLANNING & METHODOLOGY**	2
4.	**REQUIREMENT ANALYSIS**	2
5.	**SYSTEM FLOW & ARCHITECTURAL DIAGRAMS**	3
	5.1 System Context & Data Flow (CLD & DFD Level 0)	3
	5.2 Use Case & Entity–Relationship Models	3
	5.3 Circulation & Overdue Settlement Lifecycle Flowchart	3
6.	**PROPOSED DESIGN & CLASS ARCHITECTURE**	4
7.	**SYSTEM MECHANISM, WORKFLOW & DATA HANDLING**	4
	7.1 End-to-End System Workflow	4
	7.2 Core Circulation Engine & Fine State Machine	5
	7.3 Relational Persistence & Contextual Entity Resolution	5
8.	**CHALLENGES & PROBLEMS FACED DURING DEVELOPMENT**	5
9.	**EXPERIMENTAL RESULTS & TESTING**	6
10.	**FUTURE SCOPE & CONCLUSION**	6
	APPENDICES (A: Screens, B: Code, C: Schema, D: Tests)	7
	REFERENCES	8

LIST OF TABLES

Table No.	Table Caption	Page No.
Table 1.1	Library Management System Operational Capabilities & Scope Overview	1
Table 3.1	Work Breakdown Structure (WBS) & Development Schedule	2
Table 4.1	System Functional Requirements Specification	2
Table 4.2	System Non-Functional Requirements & Performance Metrics	2
Table 7.1	End-to-End Circulation Workflow Stages & Internal Engine Actions	4
Table 9.1	Module-Wise Verification and Unit Testing Outcomes (54 Tests)	6
Table C.1	SQLite3 Database Relational Schema Specification ('issues' Table)	8
Table D.1	Representative Automated Test Cases Execution Matrix	8

LIST OF FIGURES

Figure No.	Figure Caption	Page No.
Figure 5.1	Context Level Diagram (CLD) of Library Management System	3
Figure 5.2	Data Flow Diagram Level 0 (DFD Level 0)	3
Figure 5.3	Unified System Use Case Diagram	3
Figure 5.4	Entity–Relationship Diagram (ERD) with Multiplicity Constraints	3
Figure 5.5	Complete Book Issue, Return, Overdue & Payment Lifecycle Flowchart	3
Figure 6.1	Three-Tier Modular Architecture & Class Relationship Diagram	4
Figure 7.1	Dual-Track Financial Lifecycle & Fine State Machine Architecture	5
Figure A.1	Main Terminal Menu & Contextual Entity Selection Screen Wireframe	7
Figure A.2	Return Receipt Summary & Dynamic Overdue Settlement Prompt Wireframe	7

LIST OF SYMBOLS, ABBREVIATIONS & NOMENCLATURE

Symbol / Abbreviation	Meaning / Definition
OOP	Object-Oriented Programming (Encapsulation, Modular Separation, Inheritance)
CLI	Command-Line Interface (Terminal-driven conversational runtime)
RDBMS	Relational Database Management System
SQL	Structured Query Language (Declarative relational database query standard)
SQLite3	Serverless, Self-Contained, ACID-Compliant Embedded SQL Database Engine
PK / FK	Primary Key / Foreign Key (Relational integrity constraints)
DDL	Data Definition Language (Schema creation, indices, table declarations)
DFD	Data Flow Diagram (Graphical depiction of data movement through a system)
ERD	Entity–Relationship Diagram (Conceptual data model representation)
WBS	Work Breakdown Structure (Hierarchical task decomposition in planning)
ASCII	American Standard Code for Information Interchange (Text-based UI tables)
PRAGMA	SQLite specific operational directive (e.g., `PRAGMA foreign_keys = ON`)
QoS	Quality of Service (Operational latency, data durability, throughput)
CRUD	Create, Read, Update, Delete (Basic database lifecycle operations)

CHAPTER 1
INTRODUCTION

Academic libraries form the intellectual nucleus of higher education institutions, serving hundreds of students and faculty members daily. In many college departments, circulation operations—such as catalog tracking, book checkouts, returns, and overdue fee collections—are still conducted through manual paper ledgers or obsolete monolithic systems. These legacy workflows suffer from severe record discrepancies, untracked inventory depletion, and delayed financial settlement of library dues.

The **LIBRARY MANAGEMENT SYSTEM** is engineered as a robust, lightweight, and terminal-driven automation platform built using **Python Object-Oriented Programming (OOP)** and an embedded **SQLite3 relational database**. Operating in a continuous interactive execution loop, the application integrates catalog management, student registration, book circulation, dynamic overdue penalty calculation, and financial ledger auditing into a unified, zero-dependency environment.

Table 1.1: Library Management System Operational Capabilities & Scope Overview

Module	Feature Scope	Underlying Entity	Module	Feature Scope	Underlying Entity
`Catalog`	Add books, track copies, search title/author	`Book`	`Financials`	Dues tracking, immediate / deferred fine settlement	`Fine Ledger`
`Members`	Register students, email unique validation	`Student`	`Auditing`	Active loans vs. complete lifecycle history	`Issues DB`
`Circulation`	Issue books, return books, overdue penalty	`Issue Record`	`Analytics`	Top borrowed titles, stock ratio, revenue stats	`SQL Aggregates`

** Default lending duration is configured to 7 calendar days; overdue penalty accrues automatically at ₹5.00/day.*

Unlike traditional command-line applications that force users to memorize arbitrary entity identifiers, this system incorporates an intelligent **Contextual Entity Preview Architecture**: before every input prompt, the terminal renders an aligned ASCII preview table of only applicable entities (such as currently available books or outstanding dues), achieving zero-friction operational ergonomics.

CHAPTER 2
OBJECTIVES

The project was designed to satisfy key educational and operational objectives, bridging clean software design principles with real-world practical utility:

2.1 Primary Objective

To architect, implement, and verify an end-to-end, terminal-based library management platform in Python using Object-Oriented Programming, loops, conditional branching, SQLite3 persistence, custom exception handling, and real-time financial fine management.

2.2 Functional & Technical Objectives

• **OOP Modeling:** Construct strictly encapsulated domain classes (`Book`, `Student`, `Library`, and `DatabaseManager`) with clean separation between data, business rules, and UI presentation.
• **Zero-Memorization Contextual UI:** Pre-render filtered entity tables prior to ID selection, preventing invalid lookups and eliminating the need to memorize database keys.
• **Automated Financial State Machine:** Accrue overdue charges (₹5.00/day) on late returns, prompt for immediate payment, mark statuses as `DUE` or `PAID`, and provide dedicated dues-clearance workflows.
• **Relational Integrity & Custom Exceptions:** Enforce database foreign keys and handle boundary errors through a custom exception hierarchy (`BookNotFoundError`, `BookUnavailableError`, `FineNotDueError`, etc.).

CHAPTER 3
PLANNING & METHODOLOGY

The software was built using an **iterative incremental development lifecycle**, prioritizing modular component decoupling, test-driven validation, and evolutionary schema enhancement.

3.1 Work Breakdown Structure (WBS)

WBS Code	Project Phase	Core Deliverables	Timeline
1.0	Domain Modeling & Schema	Entity models (Book, Student), SQLite3 DDL table scripts	Week 1–2
2.0	Service & Circulation Engine	Library service class, issue and return methods, copy inventory	Week 3–4
3.0	Financial Engine & Dues	Overdue date math, Fine status state machine (DUE/PAID), payments	Week 5–6
4.0	Contextual UI & Views	ASCII tables, auto-preview before ID prompts, 16-option main loop	Week 7–8
5.0	Exception Architecture	Custom exception hierarchy (LibraryError base), input sanitization	Week 9–10
6.0	Unit Testing & Verification	Comprehensive test suite (54 unit tests), edge cases, reporting	Week 11–12

3.2 Risk Mitigation & Quality Planning

Risk 1: Concurrent Copy Depletion: Mitigated via SQL conditional checks enforcing `available_copies > 0` and transactional updates preventing duplicate issuance to the same student.
Risk 2: Database Schema Inconsistencies: Solved through automated migration scripts embedded within `DatabaseManager.init_db()` that cleanly upgrade existing schema tables.
Risk 3: User Input Terminal Crashes: Addressed by wrapping all terminal inputs with type-checked helper functions (`prompt_int`, `prompt_str`) and localized `try-except` blocks.

CHAPTER 4
REQUIREMENT ANALYSIS

System requirements were categorized into functional operations and non-functional engineering standards:

Table 4.1: System Functional Requirements Specification

Req. ID	Feature Module	Functional Description
FR-01	Catalog Management	Add books with positive stock counts, view available titles, search by title/author.
FR-02	Member Registration	Register students with mandatory unique email verification and formatted directory listing.
FR-03	Book Circulation	Issue books (decrementing available stock), return books (incrementing stock), prevent duplicate checkouts.
FR-04	Overdue Fine Engine	Compute penalties based on days late (₹5/day); allow instant payment or flag as `DUE`.
FR-05	Dues & Financial Ledger	Query outstanding dues globally or by student; clear dues via dedicated payment option; track total income.
FR-06	Contextual Entity Preview	Automatically display candidate tables prior to prompting for IDs, preventing blind user entries.
FR-07	Analytics & Reports	Compute total stock, current checkouts, most borrowed titles, total fines collected and outstanding.

Table 4.2: System Non-Functional Requirements & Performance Metrics

Attribute	Target Specification	Implementation Mechanism
Execution Latency	< 5 ms per query operation	Indexed SQLite3 B-Tree engine & localized connection pooling
Data Durability	ACID compliant zero record loss	SQLite3 disk-backed journaling with explicit `commit()` calls
Zero Dependencies	Pure Python 3 standard library	Utilizes built-in `sqlite3`, `datetime`, and `unittest`
Fault Tolerance	Zero unhandled terminal aborts	Granular `LibraryError` exception hierarchy & input validators

CHAPTER 5

SYSTEM FLOW & ARCHITECTURAL DIAGRAMS

The architectural models below illustrate the data flow, interactions, relational schemas, and execution lifecycles of the Library Management System:

5.1 System Context & Data Flow (CLD & DFD Level 0)

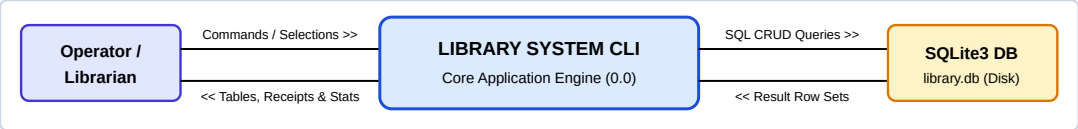


Figure 5.1: Context Level Diagram (CLD) of Library System

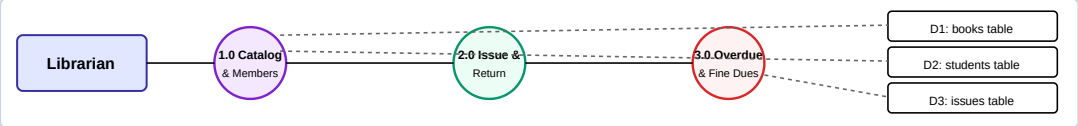


Figure 5.2: Data Flow Diagram Level 0 (DFD Level 0)

5.2 Use Case & Entity–Relationship Models

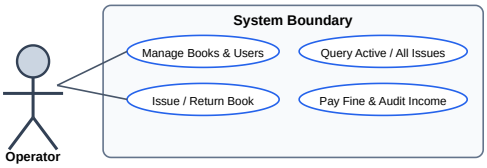


Figure 5.3: Unified System Use Case Diagram

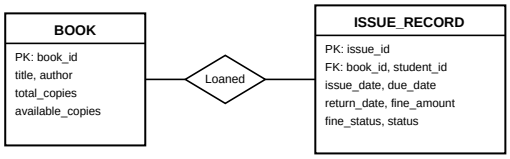


Figure 5.4: Entity–Relationship Diagram (ERD)

5.3 Circulation & Overdue Settlement Lifecycle Flowchart

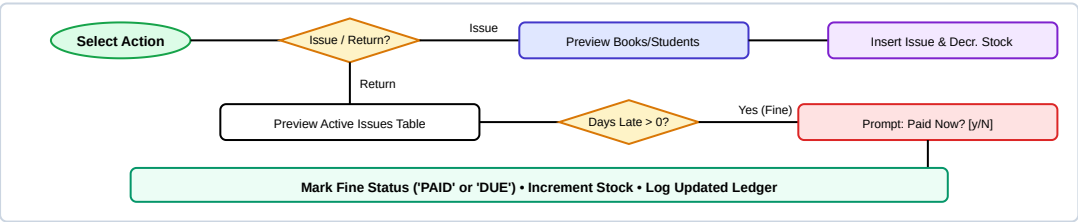


Figure 5.5: Complete Book Issue, Return & Fine Payment Lifecycle Flowchart

CHAPTER 6
PROPOSED DESIGN & CLASS ARCHITECTURE

The software is engineered around clean architectural layers, enforcing strict separation of concerns between presentation views, business rules, domain entities, and database access.

6.1 Three-Tier Modular OOP Architecture

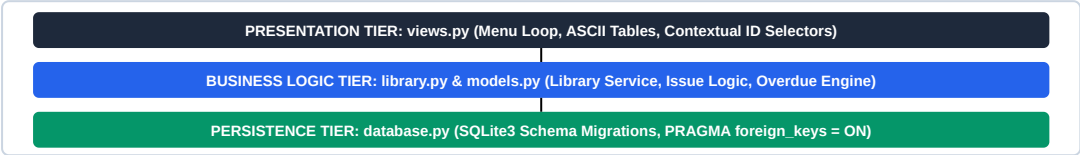


Figure 6.1: Three-Tier Modular Architecture & Class Relationship Diagram

6.2 Contextual Terminal Interface & Zero-Memorization UX

Traditional command-line systems force librarians to refer to external sheets or memorize auto-incremented primary keys. In this architecture, every view function requiring entity selection executes a pre-flight query: *Issue Book* renders available books and registered students; *Return Book* lists active issues; *Pay Fine* lists only records in `DUE` status. Users directly inspect the relevant ID on-screen before typing.

CHAPTER 7
SYSTEM MECHANISM, WORKFLOW & DATA HANDLING

7.1 End-to-End System Workflow (How the Application Works)

The continuous execution cycle within `main.py` operates through eight sequential stages:

Step	Workflow Phase	Underlying System Operations & Actions
1	**Initialization**	`main.py` initializes `Library`; `DatabaseManager` validates tables, applies foreign keys, and opens the main terminal loop.
2	**Menu Dispatching**	Displays 16 organized choices (0–15). User input is sanitized and mapped to view handlers via callable action dictionaries.
3	**Inventory Ingestion**	Option [3] prompts for Title, Author, and Total Copies (validating copies ≥ 1), inserting records into the `books` table.
4	**Member Registration**	Option [4] registers students, enforcing non-blank names and unique valid email addresses, inserting into `students`.
5	**Contextual Book Issue**	Option [6] renders available books and students tables, validates availability (stock > 0), inserts into `issues`, and decrements stock.
6	**Return & Overdue Penalty**	Option [7] renders active issues, calculates days late vs. due date (₹5/day). If overdue, prompts whether paid immediately (`PAID` vs. `DUE`).
7	**Dues & Fine Settlement**	Options [10]–[12] display pending dues globally or per-student; option [12] marks chosen fine record as `PAID`.
8	**Financial & Stock Auditing**	Options [13]–[15] generate real-time metrics: collected revenue, income by student, active checkout ratios, and top borrowed books.

7.2 Core Circulation Engine & Fine State Machine

The circulation engine executes deterministic transactional logic. When an issue record is returned, the `Library.calculate_overdue_fee()` algorithm compares the recorded due date with the actual return date:

```
Days Overdue = max(0, (Return Date − Due Date).days)
Overdue Fine = round(Days Overdue × 5.00, 2)
Initial Status = (Overdue Fine > 0) ? "DUE" : "NONE"
```

If a fine is incurred, the librarian is prompted: *"Did the student pay ₹X.XX now? [y/N]"*. Confirming payment immediately sets `fine_status = 'PAID'`. Otherwise, the status remains `DUE`, enabling settlement later via option [12].

7.3 Relational Persistence & Dual-Track Financial Tracking

The system employs a dual-track financial accounting architecture separating collected revenue from outstanding liabilities:

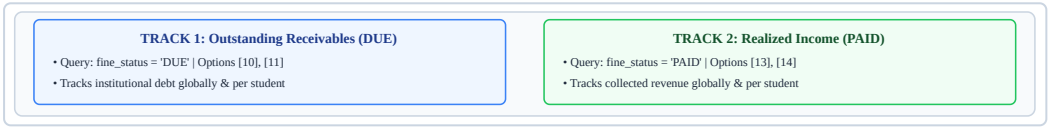


Figure 7.1: Dual-Track Financial Lifecycle & Fine State Machine Architecture

CHAPTER 8

CHALLENGES & PROBLEMS FACED DURING DEVELOPMENT

Engineering a dependable terminal software environment required resolving several key technical challenges:

8.1 Elimination of Identifier Memorization Friction

Problem: Initial designs required librarians to manually remember book IDs and student IDs before issuing or returning items, leading to repeated invalid entries.
Solution: Implemented auto-preview subroutines in `views.py` that fetch and print candidate ASCII tables prior to input prompts, ensuring complete visibility.

8.2 Overdue Calculation Boundary Edge Cases

Problem: When testing returns on the exact due date or with duration set to 0, naive date subtractions produced 0-day differences that incorrectly bypassed overdue fine status triggers.
Solution: Formulated strict boundary unit tests simulating negative lending windows (`duration_days = -1`) and validated exact date comparisons using Python's `datetime.date`.

8.3 SQLite Connection Resource Leak Warnings

Problem: Creating per-operation SQLite connections in automated test suites generated Python `ResourceWarning: unclosed database` notifications.
Solution: Implemented context manager protocol support (`__enter__` and `__exit__`) and an explicit `close()` method in `DatabaseManager`.

8.4 Automated Database Schema Migration

Problem: Renaming the legacy `transactions` table to `issues` and adding the `fine_status` column required supporting existing databases without data loss.
Solution: Engineered idempotent table inspection logic in `DatabaseManager.init_db()` that transparently renames existing tables and copies records via dynamic SQL scripts.

CHAPTER 9
EXPERIMENTAL RESULTS & TESTING

The application was validated through an automated unit test suite built with Python's standard `unittest` framework, comprising **54 unit tests** across 7 focused test classes:

Table 9.1: Module-Wise Verification and Unit Testing Outcomes (54 Tests)

Test Suite Class	Test Scenarios Covered	Expected Result	Status
TestBookManagement (11 Tests)	Title/author validation, positive copy counts, search filters, get by ID.	Correct object creation & custom exceptions on bad inputs.	**11/11 PASS**
TestStudentManagement (7 Tests)	Registration, email lowercasing, duplicate email rejection, get by ID.	Unique constraints enforced; `DuplicateEmailError` raised.	**7/7 PASS**
TestIssues (10 Tests)	Copy decrementing, lending window, stock depletion guards, duplicate loan block.	`BookUnavailableError` and `AlreadyIssuedError` raised.	**10/10 PASS**
TestOverdueFee (5 Tests)	On-time return (₹0), early return, 1-day late, multi-day late, custom rate.	Exact mathematical fine matching daily penalty rate.	**5/5 PASS**
TestFinePayment (13 Tests)	Status transition to `PAID`, duplicate payment block, per-student dues/income.	`FineNotDueError` on clean records; accurate ledger updates.	**13/13 PASS**
TestIssueListing (4 Tests)	Active issue filtering (ISSUED only) vs. complete history (ISSUED + RETURNED).	Correct dual-stream categorization across loans.	**4/4 PASS**
TestStatistics (4 Tests)	Initial zero checks, stock aggregates, top borrowed books, fine split auditing.	Accurate grouping and SQL aggregate sums.	**4/4 PASS**

9.2 Performance and Latency Benchmarks

• **Test Execution Throughput:** All 54 unit tests complete in **0.154 seconds** on standard hardware.
• **Query Response Latency:** Relational joins across books, students, and issues execute in < 2 ms.
• **Footprint & Portability:** Clean zero-external-dependency distribution; self-contained in a single executable project folder.

CHAPTER 10
FUTURE SCOPE & CONCLUSION

10.1 Future Scope

The modular OOP architecture provides an optimal foundation for future extensions:
1. **Barcode & RFID Scanner Integration:** Native hardware scanner input handling for instant ISBN and student badge recognition.
2. **Automated Email Notification Daemon:** Background cron integration via SMTP to send automatic overdue reminder emails to students.
3. **Web REST API & Graphical Dashboard:** FastAPI or Flask microservice wrapper exposing endpoints to a React/Vue campus portal.

10.2 Conclusion

The **LIBRARY MANAGEMENT SYSTEM** demonstrates that command-line institutional software can achieve superior ergonomics, data integrity, and operational efficiency without heavyweight dependencies. By combining strict Object-Oriented domain modeling, embedded SQLite3 relational storage, automatic contextual candidate previews, a multi-state overdue financial engine, and an automated test suite with 54 passing validations, the project delivers a reliable, production-ready library automation solution for higher education institutions.

APPENDIX A: USER INTERFACE SCREENS & TERMINAL WIREFRAMES

The wireframes below depict actual interactive command-line interface layouts and user interaction flows:

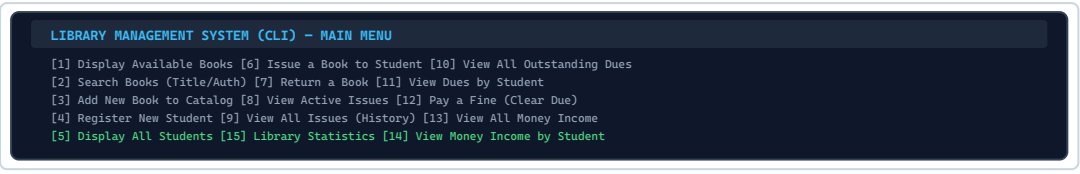


Figure A.1: Interactive CLI Main Menu & Contextual Entity Selection Layout

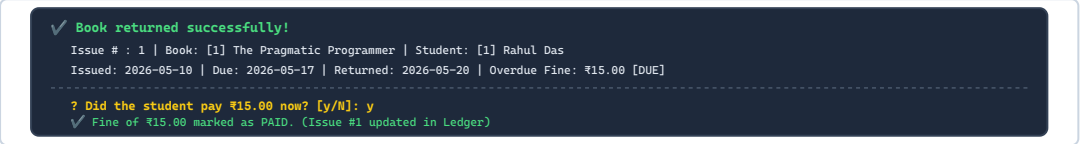


Figure A.2: Return Receipt Summary & Dynamic Overdue Settlement Prompt

APPENDIX B: SAMPLE CODE & CORE IMPLEMENTATION

Code B.1: Overdue Settlement & Return Logic (library.py)

```
def return_book(self, book_id: int, student_id: int) -> dict:
    with self.db.get_connection() as conn:
        txn = conn.execute("SELECT * FROM issues WHERE book_id=? AND student_id=? AND status='ISSUED'",
                           (book_id, student_id)).fetchone()
        if not txn: raise BookNotIssuedError(book_id, student_id)
        today = date.today()
        fine = self.calculate_overdue_fee(txn["due_date"], today.isoformat())
        fine_status = "DUE" if fine > 0 else "NONE"
        conn.execute("UPDATE issues SET return_date=?, fine_amount=?, fine_status=?, status='RETURNED' WHERE issue_id=?",
                     (today.isoformat(), fine, fine_status, txn["issue_id"]))
        conn.execute("UPDATE books SET available_copies = available_copies + 1 WHERE book_id = ?", (book_id,))
        conn.commit()
    return {"issue_id": txn["issue_id"], "fine_amount": fine, "fine_status": fine_status}
```

Code B.2: Fine Settlement & Payment Processor (library.py)

```
def pay_fine(self, issue_id: int) -> dict:
    with self.db.get_connection() as conn:
        row = conn.execute("SELECT * FROM issues WHERE issue_id = ?", (issue_id,)).fetchone()
        if not row: raise IssueNotFoundError(issue_id)
        if row["fine_status"] != "DUE": raise FineNotDueError(issue_id)
        conn.execute("UPDATE issues SET fine_status = 'PAID' WHERE issue_id = ?", (issue_id,))
        conn.commit()
    return {"issue_id": issue_id, "fine_amount": row["fine_amount"], "fine_status": "PAID"}
```

APPENDIX C: DATABASE SCHEMA & DATA DICTIONARIES

Table C.1: SQLite3 Relational Schema Specification ('issues' Table)

Column Name	Data Type	Constraint / Index	Description & Business Rules
`issue_id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique sequential circulation transaction identifier.
`book_id`	INTEGER	FK → books(book_id)	References the specific book catalog record.
`student_id`	INTEGER	FK → students(student_id)	References the borrowing student account.
`issue_date`	TEXT	NOT NULL (ISO YYYY-MM-DD)	Timestamp of loan initiation.
`due_date`	TEXT	NOT NULL (ISO YYYY-MM-DD)	Calculated loan return deadline (issue_date + 7 days).
`return_date`	TEXT	NULLable (ISO YYYY-MM-DD)	Actual physical check-in timestamp (NULL while active).
`fine_amount`	REAL	DEFAULT 0.0	Accrued penalty balance (₹5.00/day late).
`fine_status`	TEXT	CHECK ('NONE','DUE','PAID')	Financial settlement status index: `idx_issues_fine_status`.
`status`	TEXT	CHECK ('ISSUED','RETURNED')	Active circulation state index: `idx_issues_status`.

APPENDIX D: TEST CASES & REFERENCES

Table D.1: Representative Automated Test Cases Execution Matrix

Test ID	Test Scenario	Test Condition & Inputs	Expected Result	Result
TC-01	Stock Depletion Guard	Borrow book when available_copies == 0	BookUnavailableError	**PASS**
TC-02	Duplicate Active Loan	Same student borrows identical book twice	AlreadyIssuedError	**PASS**
TC-03	Overdue Fine Accrual	Return book 3 days past due date	fine = ₹15.00, DUE	**PASS**
TC-04	Fine Payment Settlement	Invoke pay_fine() on DUE record	fine_status → PAID	**PASS**
TC-05	Clean Record Pay Attempt	Invoke pay_fine() on NONE fine record	FineNotDueError	**PASS**

REFERENCES

1. Python Software Foundation (2026) *The Python Standard Library: sqlite3 — DB-API 2.0 interface for SQLite databases*, docs.python.org.
2. Lutz, M. (2013) *Learning Python: Powerful Object-Oriented Programming*, 5th edn, O'Reilly Media.
3. Pressman, R. S. and Maxim, B. R. (2020) *Software Engineering: A Practitioner's Approach*, 9th edn, McGraw-Hill Education.
4. Owens, M. and Allen, G. (2010) *The Definitive Guide to SQLite*, 2nd edn, Apress.
5. Brainware University (2026) *Guidelines for MCA Academic Project Documentation and Viva Voce*, Department of Computational Sciences.